

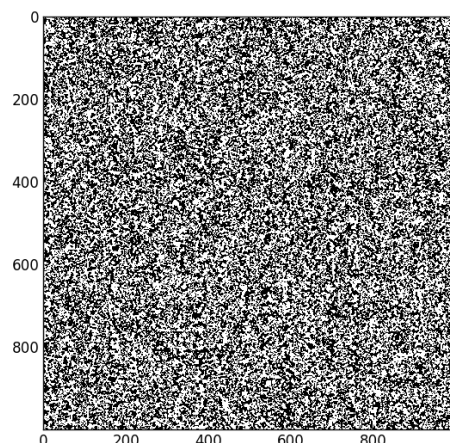
Ising models and Numba

This work is supported by Continuum Analytics (<http://continuum.io>) and the XDATA Program (<http://www.darpa.mil/program/XDATA>) as part of the Blaze Project (<http://blaze.pydata.org>)

tl;dr I play with Numba and find it effective.

Introduction

Confession, I've never actually used Numba. Well that's not quite true; I've indirectly used Numba thousands of times because Blaze auto-generates numba ufuncs (<http://continuum.io/blog/blaze-and-numba>). Still I've never used it for a particular problem. I usually define problems with large array operations and compile those down. Numba takes a different approach and translates Python for loops to efficient LLVM code. This is all lower in the hardware stack than where I usually think.



But when I was looking for applications to motivate recent work in nearest-neighbor communications in dask (<http://dask.readthedocs.org/en/latest/ghost.html>) a friend (<https://github.com/maxhutch/>) pointed me towards the Ising model (http://en.wikipedia.org/wiki/Ising_model), a simple physical system that is both easy to code up and has nice macro-scale properties. I took this as an example to play with Numba. This post details my experience.

Ising Model

Disclaimer: I am not a physicist

The Ising model represents a regular grid of points where each point has two possible states, spin up and spin down. States like to have the same spin as their immediate neighbors so when a spin-down state is surrounded by more spin-up states it will switch to spin-up and vice versa. Also, due to random fluctuations, points might switch spins, even if this switch is not favorable. In pseudocode an Ising update step might look like the following

```
for every point in the grid:
    energy = my spin * sum of all of the spins (+1 or -1) of neighboring points
    if energy is improved by switching:
        switch
    else if we're particularly unlucky
        switch anyway
```

For this kind of algorithm imperative for-loopy code is probably the most clear. You can do this with high-level array operations too (e.g. NumPy/Blaze/Theano), but it's a mess.

Numba code

Here is my solution to the problem with Numba and a gif of the solution

```
import numba
import numpy as np
from math import exp, log, e, sqrt

kT = 2 / log(1 + sqrt(2), e)

@numba.jit(nopython=True)
def _update(x, i, j):
    n, m = x.shape
    dE = 2* x[i, j] * (
        x[(i-1)%n, (j-1)%m]
        + x[(i-1)%n, j]
        + x[(i-1)%n, (j+1)%m]

        + x[i, (j-1)%m]
        + x[i, (j+1)%m]

        + x[(i+1)%n, (j-1)%m]
        + x[(i+1)%n, j]
        + x[(i+1)%n, (j+1)%m]
    )

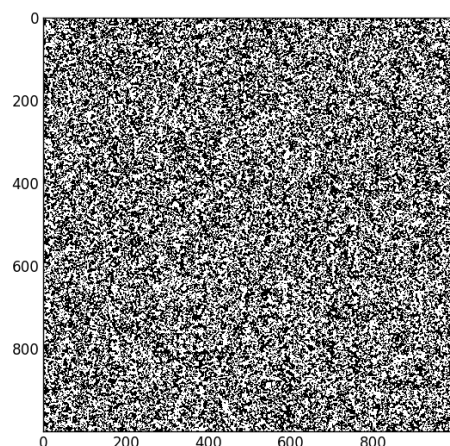
    if dE <= 0 or exp(-dE / kT) > np.random.r
andom():
        x[i, j] *= -1

@numba.jit(nopython=True)
def update(x):
    n, m = x.shape

    for i in range(n):
        for j in range(0, m, 2): # Even colu
            mns first to avoid overlap
            _update(x, i, j)

    for i in range(n):
        for j in range(1, m, 2): # Odd colum
            ns second to avoid overlap
            _update(x, i, j)

if __name__ == '__main__':
    x = np.random.randint(2, size=(1000, 100
0)).astype('i1')
    x[x == 0] = -1
    for i in range(100):
        update(x)
```



My old beater laptop executes one update step on a 1000x1000 grid in 50ms. Without Numba this takes 12s. This wasn't a canned demo by an expert user / numba developer; this was just my out-of-the-box experience.

Thoughts

I really like the following:

- I can remove `@numba.jit` and use the Python debugger
- I can assert that I'm only using LLVM with `nopython=True`
- I can manage data with NumPy (or `dask.array`) separately from managing computation with Numba

I ran in to some issues and learned some things too:

- `random` is only present in the developer preview builds of Numba (`conda install -c numba numba`). It will be officially released in the 0.18 version this March.
- You don't have to provide type signature strings. I tried providing these at first but I didn't know the syntax and so repeatedly failed to write down the type signature correctly. Turns out the cost of not writing it down is that Numba will jit whenever it sees a new signature. For my application this is essentially free.

0 Comments

 1 [Login](#) ▼

G

Start the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS 

Name



Share

Best Newest Oldest

Be the first to comment.

[Subscribe](#)

[Privacy](#)

[Do Not Sell My Data](#)

© Matthew Rocklin (<https://matthewrocklin.com/>) 2020. Atom Feed (<https://mrocklin.github.io/blog/atom.xml>)